자료구조 - 스택(Stack) 전체 소스 코드 및 수업 자료
1. 스택(Stack)이란?

스택(Stack)은 데이터를 쌓아 올리는 형태의 자료구조이다.

가장 마지막에 들어간 데이터가 가장 먼저 나오는 구조이며, 이를 LIFO(Last In First Out)라고 한다.

예를 들어 접시를 쌓아 놓았을 때 가장 마지막에 올려놓은 접시를 가장 먼저 꺼내는 것과 같은 원리이다.

스택의 주요 기능은 다음과 같다.

PUSH : 데이터를 삽입하는 기능
POP : 가장 위에 있는 데이터를 삭제하는 기능
PEEK : 가장 위에 있는 데이터를 확인하는 기능
DISPLAY : 스택에 저장된 데이터를 출력하는 기능

2. 스택 전체 소스 코드
c
```
#include <stdio.h>    // printf() 함수를 사용하기 위한 헤더 파일

#define SIZE 5         // 스택의 크기를 5로 설정

int stack[SIZE];       // 크기가 5인 정수형 배열 선언
int top = -1;          // 스택의 가장 위쪽 위치를 나타내는 변수
                       // -1은 스택이 비어 있다는 의미


// ======================================
// 1. PUSH 함수 : 데이터 삽입하기
// ======================================
void push(int data) {

    // top이 SIZE - 1이면 스택이 가득 찬 상태
    if (top == SIZE - 1) {
        printf("스택이 가득 찼습니다.\n");
    }
    else {
        top++;                // top을 1 증가시킴
        stack[top] = data;    // 해당 위치에 데이터 저장
```

```
    }
}


// ====================================
// 2. POP 함수 : 데이터 삭제하기
// ====================================
void pop() {

    // top이 -1이면 스택이 비어 있는 상태
    if (top == -1) {
        printf("스택이 비어 있습니다.\n");
    }
    else {
        // 가장 위에 있는 데이터 출력
        printf("%d 삭제 완료\n", stack[top]);

        top--;  // top을 1 감소시켜 데이터 삭제 처리
    }
}


// ====================================
// 3. PEEK 함수 : 가장 위의 데이터 확인하기
// ====================================
void peek() {

    // 스택이 비어 있는지 확인
    if (top == -1) {
        printf("스택이 비어 있습니다.\n");
    }
    else {
        // stack[top]에 저장된 가장 위의 데이터 출력
        // 데이터를 삭제하지는 않음
        printf("가장 위에 있는 데이터 : %d\n", stack[top]);
    }
}


// ====================================
// 4. DISPLAY 함수 : 데이터 출력하기
```

```
// ====================================
void display() {

    int i;  // 반복문에서 사용할 변수

    // 스택이 비어 있는지 확인
    if (top == -1) {
        printf("스택이 비어 있습니다.\n");
    }
    else {

        // 가장 위에 있는 위치부터 0까지 반복
        for (i = top; i >= 0; i--) {

            // 배열에 저장된 데이터 출력
            printf("%d ", stack[i]);
        }

        printf("\n");  // 줄 바꿈
    }
}


// ====================================
// 5. MAIN 함수 : 프로그램 실행하기
// ====================================
int main() {

    push(10);  // 스택에 10 삽입
    push(20);  // 스택에 20 삽입
    push(30);  // 스택에 30 삽입

    display(); // 스택에 저장된 데이터 출력

    return 0;  // 프로그램 정상 종료
}
```

3. 각 함수의 역할 이해하기

① 스택 선언하기

c

```
#define SIZE 5
```

```
int stack[SIZE];
int top = -1;
```

#define SIZE 5 : 스택의 크기를 5로 설정한다.
int stack[SIZE] : 정수형 데이터를 저장할 배열을 선언한다.
int top = -1 : 스택이 비어 있다는 것을 나타낸다.

스택의 배열 인덱스는 0부터 4까지 사용한다.

처음에는 데이터가 없기 때문에 top의 값은 -1이다.

② PUSH 함수 - 데이터 삽입하기

```
c
void push(int data) {

    if (top == SIZE - 1) {
        printf("스택이 가득 찼습니다.\n");
    }
    else {
        top++;
        stack[top] = data;
    }
}
```

PUSH는 스택에 데이터를 삽입하는 기능이다.

예를 들어 다음과 같이 실행한다.

```
c
push(10);
push(20);
push(30);
```

실행 과정은 다음과 같다.

push(10) 실행 → 10 삽입, top은 0
push(20) 실행 → 20 삽입, top은 1
push(30) 실행 → 30 삽입, top은 2

핵심 코드

top++ : top의 값을 1 증가시킨다.
stack[top] = data : 해당 위치에 데이터를 저장한다.
③ POP 함수 - 데이터 삭제하기
c
```
void pop() {

    if (top == -1) {
        printf("스택이 비어 있습니다.\n");
    }
    else {
        printf("%d 삭제 완료\n", stack[top]);
        top--;
    }
}
```

POP은 가장 마지막에 삽입된 데이터를 삭제하는 기능이다.

예를 들어 스택에 10, 20, 30이 들어 있다면 POP을 실행했을 때 30이 삭제된다.

stack[top] : 가장 위에 있는 데이터
top-- : top의 값을 1 감소시킨다.

참고로 실제 배열의 값을 지우는 것이 아니라 top의 위치를 감소시켜 삭제된 것처럼 처리한다.

④ PEEK 함수 - 가장 위의 데이터 확인하기
c
```
void peek() {

    if (top == -1) {
        printf("스택이 비어 있습니다.\n");
    }
    else {
        printf("가장 위에 있는 데이터 : %d\n", stack[top]);
    }
}
```

PEEK은 가장 위에 있는 데이터를 확인하는 기능이다.

예를 들어 스택에 10, 20, 30이 들어 있다면 다음과 같이 출력된다.

text
가장 위에 있는 데이터 : 30

PEEK은 데이터를 삭제하지 않는다.

⑤ DISPLAY 함수 - 데이터 출력하기
c
```c
void display() {

    int i;

    if (top == -1) {
        printf("스택이 비어 있습니다.\n");
    }
    else {
        for (i = top; i >= 0; i--) {
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}
```

DISPLAY는 스택에 저장된 모든 데이터를 출력하는 기능이다.

가장 위에 있는 데이터부터 출력한다.

핵심 코드

c
```c
for (i = top; i >= 0; i--)
```

i = top : 가장 위에 있는 위치부터 시작한다.
i >= 0 : 0번 위치까지 반복한다.
i-- : i의 값을 1씩 감소시킨다.

10, 20, 30을 삽입했다면 출력 결과는 다음과 같다.

text
30 20 10

⑥ MAIN 함수 - 프로그램 실행하기
c
```
int main() {

    push(10);
    push(20);
    push(30);

    display();

    return 0;
}
```

main 함수는 프로그램이 시작되는 부분이다.

실행 순서는 다음과 같다.

push(10) : 10 삽입
push(20) : 20 삽입
push(30) : 30 삽입
display() : 저장된 데이터 출력

실행 결과

text
30 20 10

4. 반드시 기억해야 할 핵심 내용

구분	의미	핵심 코드
PUSH	데이터 삽입	top++
POP	데이터 삭제	top--
PEEK	가장 위의 데이터 확인	stack[top]
DISPLAY	데이터 전체 출력	for (i = top; i >= 0; i--)

추가로 다음 내용을 반드시 기억하자.

스택이 비어 있는 상태 : top == -1
스택이 가득 찬 상태 : top == SIZE - 1
스택의 크기가 5일 때 마지막 인덱스 : 4
스택의 데이터 저장 방식 : LIFO(Last In First Out)
5. 이해도 확인 문제

문제 1.

다음 코드를 실행한 후 DISPLAY를 실행하면 어떤 결과가 출력될까?

```
c
push(10);
push(20);
push(30);
display();
```

정답 : 30 20 10

문제 2.

10, 20, 30을 삽입한 뒤 POP을 실행하면 삭제되는 데이터는?

정답 : 30

문제 3.

스택이 비어 있을 때 top의 값은?

정답 : -1

문제 4.

PEEK 함수를 실행하면 데이터는 어떻게 될까?

정답 : 가장 위에 있는 데이터만 확인하고 삭제하지 않는다.

문제 5.

스택의 크기가 5일 때 스택이 가득 찬 상태의 top 값은?

정답 : 4

6. 최종 정리

스택의 핵심은 네 가지 함수의 역할을 이해하는 것이다.

PUSH : 데이터를 넣는다.
POP : 가장 위의 데이터를 삭제한다.
PEEK : 가장 위의 데이터를 확인한다.
DISPLAY : 저장된 데이터를 출력한다.

단순히 코드를 암기하지 말고 각 함수에서 top이 어떻게 변화하는지, stack[top]이 무엇을 의미하는지 이해하도록 하자.

수행평가에서는 전체 코드의 구조와 각 함수의 역할을 이해하고 있는지 확인한다.